CENTRALIZED ORCHESTRATION, REGIONAL EXECUTION A PLACEMENT PATTERN FOR LATENCY-SENSITIVE CLOUD PIPELINES

SHAYAN GHASEMNEZHAD, GIACINTO PAOLO (GP) SAGGESE, PAUL SMITH, AND HEANH SOK

Abstract. Workflow schedulers usually live in one region. The work they manage often belongs somewhere else. For latency-sensitive pipelines, that mismatch is not a minor deployment detail. It becomes part of the freshness budget. Teams usually pay in one of two ways: keep one scheduler and launch tasks across the wide-area network, or run one scheduler per region and carry the cost of duplicated deployments, diverging operators, and synchronization glue.

We describe a third pattern: one scheduler, regional execution. The scheduler remains a single control plane. Each workflow declares its region through a deterministic placement contract, so placement is fixed before the workflow runs. Tasks launch as containers in the target region through Amazon ECS and Fargate. A bounded retry layer catches known cloud-launch failures such as capacity unavailability, network-interface provisioning delays, resource-initialization failures, and ephemeral-storage provisioning delays. The scheduler sees bounded delay or a clean terminal result, not a storm of substrate flakes.

We report a production implementation in the Causify Trading Platform using Airflow on Kubernetes as the control plane and ECS/Fargate as the elastic regional execution plane. The repository contains 69 region-tagged DAG files across Tokyo and Europe, with 29,401 implied scheduled triggers per day from periodic pre-production workflows alone. A representative exchange API measurement showed Tokyo callers 12.7 ms faster than Singapore callers for a Crypto.com order-book endpoint; Tokyo-to-Singapore RTT measured about 68.8 ms, with representative TCP throughput around 190 Mbit/s. The numbers are not universal. They make the operational point: geography leaks into pipeline freshness, and the right abstraction makes placement explicit, reviewable, and cheap to operate.

Keywords. workflow orchestration; geo-distributed pipelines; latency-sensitive systems; Airflow; ECS; Fargate; cloud scheduling; near-exchange compute; regional execution; bounded retries; data freshness

Contents

Date: July 2026.

1. Introduction

Latency-sensitive pipelines are allergic to geography. A model is only as fresh as its inputs. In trading, freshness competes directly with everyone else's freshness. The uncomfortable part is that the compute is often not the bottleneck. The path is. Network distance, queueing, cloud task launch, container startup, database commit, and retry behavior all become part of the product.

Traditional high-frequency trading attacks the problem with co-location and private networks. Cloud-native platforms rarely get that luxury. They get regions. That is enough to turn the problem into a placement problem: run collectors and short-lived jobs near the exchange endpoint, near the data source, or near the storage surface they update, while keeping the scheduler boring and centralized.

The trap is operational. A scheduler in every region sounds clean until the copies drift. DAG distribution diverges. Operator versions differ. Upgrades become regional events. Cross-region workflows need coordination. And the thing meant to reduce latency turns into a new source of operational drag.

Causify Trading Platform uses a different split. Airflow is the single scheduler and control plane. Tasks run as containers in the region encoded by the workflow itself. The handoff to cloud execution is where region, network configuration, storage mounts, task sizing, and retry semantics are resolved. The scheduler does not guess at runtime and does not rely on a README convention. Placement is a contract.

Contributions.

- A production pattern for latency-sensitive geo-distributed pipelines with one scheduler and regional execution.
- Two named mechanisms: a *deterministic placement contract*, which binds each workflow to a region before scheduling, and *bounded reliability semantics*, which turns known cloud-launch flakes into bounded delay.
- A concrete Airflow, Kubernetes, ECS, and Fargate implementation, including region-aware task launch, sanitized code examples, measured network baselines, and a repository-derived DAG inventory.
- A freshness metric, *knowledge latency*, that measures when a data point becomes query-visible instead of pretending that raw network RTT is the whole story.

2. The architectural tension

Most geo-distributed pipeline deployments drift toward one of two designs.

Design A. One scheduler, one region. A single scheduler runs in a single region. This keeps operations simple: one DAG distribution path, one scheduler state, one upgrade surface. The price is paid by workloads. Every task that belongs near a remote source launches across region boundaries or talks across the WAN.

Design B. One scheduler per region. A scheduler runs in every region. This keeps local tasks local, but the operational surface grows with every region. Schedulers drift apart, DAG distribution is duplicated, operator versions diverge, and workflows that span regions need additional coordination.

Neither design is absurd. One pays in latency. The other pays in operations. The trade-off appears fundamental only because two concerns are coupled: where scheduling state lives and where work executes. Separating them gives a cleaner design: keep one scheduler, run each task in the region where it belongs.

2.1. Geography is not an implementation detail.

Cross-region latency is bounded by propagation delay, path length, and network topology. Engineering can improve routing and remove avoidable overhead, but it cannot make Tokyo and Singapore occupy the same point in space [21, 10].

For a pipeline that ingests from a remote source, the end-to-end time from data arrival to query visibility is roughly

$$T \approx T_{\text{network}} + T_{\text{collector}} + T_{\text{queue}} + T_{\text{db}} + T_{\text{availability}}.$$

When the collector sits in the wrong region, both T_{network} and often T_{queue} grow. The rest of the architecture can be elegant and still lose freshness through placement.

2.2. **The third design.** The third design is simple enough to sound obvious after the fact: one scheduler, many execution regions. Airflow parses and schedules workflows in one place. A workflow's region is resolved from its identity before it runs. The launch factory asks ECS/Fargate to run the container in that region. Known launch failures are retried below the scheduler with fixed budgets.

The result is not exchange co-location. It is a cloud-native approximation of the same instinct: put short-lived work close to the thing it serves, and do it without multiplying the scheduler.

3. Design principles

Table 1 summarizes the pattern. These principles are intentionally technology-neutral; Airflow and ECS/Fargate are one implementation.

Principle	**Commitment**
P1 One scheduler	A single scheduler owns workflow state, dependency ordering, and DAG distribution.
P2 Placement from identity	Each workflow's region is part of its identity and is resolved before scheduling.
P3 Elastic regional execution	Tasks launch as regional containers on demand; no permanent worker fleet is required in each region.
P4 Bounded launch retries	Known cloud-launch failures are retried with fixed budgets below the scheduler.
P5 Freshness as the SLO	The metric is end-to-end time from data arrival to query visibility, not raw network RTT.

Table 1. The five design principles behind centralized orchestration and regional execution.

3.1. **P1. One scheduler.** A single scheduler runs all workflows. This rules out per-region scheduler fleets and scheduler-to-scheduler synchronization. It commits the platform to one source of truth for workflow state, one operator version set, and one upgrade path.

3.2. **P2. Placement from identity.** A *deterministic placement contract* ties each workflow to a region and its associated network and storage configuration. In our implementation the contract lives in the workflow filename. Other systems could encode the same contract in a typed manifest, a Kubernetes custom resource, or a workflow label validated at admission time.

The important part is not the filename. The important part is that region selection is fixed before execution. Choosing a region at trigger time, passing region flags through ad-hoc operator arguments, or documenting placement in a README leaves too much room for quiet drift.

3.3. **P3. Elastic regional execution.** Tasks launch as containers in the target region and disappear when they finish. Region cost scales with workload. The system avoids keeping a permanent scheduler and worker pool alive in every region just to catch bursts.

3.4. **P4. Bounded reliability semantics.** Cloud launch failures are real and usually boring. Capacity is unavailable. A network interface takes too long to provision. Resource initialization fails. Ephemeral storage provisioning times out. A good pipeline should not turn those known substrate failures into manual recovery work every time.

bounded reliability semantics classifies launch failures into three groups:

(1) recoverable at launch, such as temporary capacity unavailability;
(2) recoverable just after launch, such as network-interface or ephemeral-storage provisioning failures;
(3) terminal, where retrying only hides a real error.

Recoverable cases receive bounded retries. Terminal cases fail clearly. The scheduler sees a clean outcome or a bounded delay, not unbounded flakiness.

3.5. **P5. Freshness as the metric.** We define *knowledge latency* K as

$$K = t_{\text{commit}} - t_{\text{receive}},$$

where t_{receive} is the timestamp when a data point enters collector memory and t_{commit} is when the corresponding record becomes query-visible. This captures transport, buffering, serialization, database write, retry cost, and final availability. It measures the thing users actually care about: when the system knows.

4. Architecture

4.1. **Two planes and one handoff.** The system has a control plane, a handoff point, and an execution plane. The control plane runs Airflow, parses DAG files, resolves placement, and asks the cloud to launch work. The execution plane is a set of regional containers and per-region storage surfaces. The handoff point is the launch factory, where the placement contract becomes concrete cloud configuration.

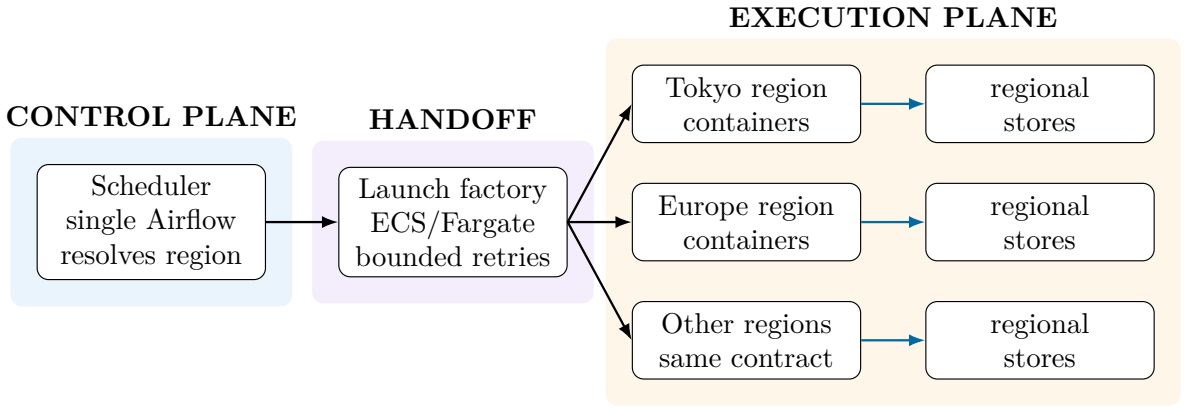

Figure 1. A single scheduler resolves placement before launch. The handoff point converts workflow identity into region, cluster, network, storage, and retry behavior. Regional containers execute the work and write to regional stores.

4.2. **The handoff contract.** The handoff has three properties.

- **Region is already chosen.** The cloud does not pick a region on behalf of the workflow. The workflow carries its region through the placement contract.
- **Failures are labeled.** Known launch and post-launch provisioning failures are classified before they reach the scheduler as ordinary task failures.

- **The scheduler does not retry substrate flakes.** Substrate retries happen inside the launch wrapper, with fixed budgets. Airflow receives the bounded result.

4.3. **End-to-end flow.** A run proceeds as follows. Airflow parses a workflow. The shared parser extracts the stage, location, purpose, period, dataset information, and optional tags from the filename. The location token maps to an AWS region, region-specific networking, and storage mounts. The launch factory builds an ECS/Fargate task invocation for that region. Known launch failures are retried within budget. The task writes to regional storage or the appropriate database surface. Cross-region movement, when needed, is modeled as its own workflow rather than being hidden inside a placement side effect.

5. Implementation

5.1. **Airflow on Kubernetes as the control plane.** The control plane is a single Airflow deployment on Kubernetes. The scheduler, webserver, and DAG processor run as Kubernetes deployments. DAG files are distributed through `git-sync`. The Airflow UI sits behind an internal ingress with TLS. Horizontal Pod Autoscaling handles control-plane pressure, while cleanup jobs remove old pods and logs. Kubernetes supplies the scheduler's operating surface; ECS/Fargate supplies regional execution.

This split is deliberate. The scheduler remains stable. Regional execution remains elastic. The control plane knows about regions only through placement contract resolution.

5.2. **Filename-as-configuration.** The production parser treats workflow names as structured configuration. A location token such as `tokyo` or `europe` maps to an AWS region and storage mount before scheduling. Listing 1 shows the sanitized core.

```python
EUROPE_REGION = "eu-north-1"
ASIA_REGION = "ap-northeast-1"
valid_locations = ["tokyo", "europe"]

def extract_components_from_filename(filename: str, dag_type: str) -> Dict:
    schema = schema_for(dag_type)
    components = filename.split(".")[:-1]
    values = {schema[i]: components[i] for i in range(len(components))}
    assert values["location"] in valid_locations
    values["region"] = (
        ASIA_REGION if values["location"] == "tokyo" else EUROPE_REGION
    )
    values["efs_mount"] = (
        "{{ var.value.efs_mount_tokyo }}"
        if values["region"] == ASIA_REGION
        else "{{ var.value.efs_mount }}"
    )
    values["dag_id"] = ".".join(components)
    return values
```

Listing 1. Sanitized deterministic placement contract. Workflow identity resolves to region and storage at parse time.

The parser is intentionally boring. That is its virtue. A placement change is a filename and code-review event, not a flag someone can flip at runtime without leaving a visible trail.

5.3. **A region-aware launch factory.** All regional execution passes through a single launch factory. It builds an Airflow `EcsRunTaskOperator` for the target region and fills in the ECS cluster, networking, task sizing, storage, logging, and timeout. Listing 2 shows a sanitized version.

```python
def get_ecs_run_task_operator(
    *, region="eu-north-1", assign_public_ip=False,
    ephemeral_storage_gb=21, launch_type="fargate", task_cmd=None
):
    network_configuration = {
        "awsvpcConfiguration": {
            "securityGroups": sg_for(region, public=assign_public_ip),
            "subnets": subnets_for(region, public=assign_public_ip),
            "assignPublicIp": "ENABLED" if assign_public_ip else "DISABLED",
        }
    }
    return EcsRunTaskOperator_withRetry(
        region=region,
        cluster=get_ecs_cluster(stage, region),
        task_definition=task_definition,
        launch_type=launch_type.upper(),
        network_configuration=network_configuration,
        overrides={
            "cpu": str(cpu_units),
            "memory": str(memory_mb),
            "ephemeralStorage": {"sizeInGiB": ephemeral_storage_gb},
            "containerOverrides": [{"name": task_definition, "command": task_cmd}],
        },
        awslogs_group=f"/ecs/{task_definition}",
        awslogs_stream_prefix=f"ecs/{task_definition}",
        waiter_max_attempts=1000000,
        execution_timeout=datetime.timedelta(hours=26),
    )
```

LISTING 2. Sanitized launch factory. Region decides cluster, networking, task sizing, and log group.

The large waiter bound is not a claim that jobs should run forever. It prevents the Airflow operator from giving up before long-running regional jobs complete. Real timeout behavior is still governed by task-level execution budgets and the surrounding workflow.

5.4. **Bounded retries for the failures we actually see.** The ECS wrapper adds two retry points. The first surrounds `RunTask` and handles temporary capacity failures. The second wraps task success checking and retries known post-launch provisioning failures. Listing 3 shows the sanitized structure.

```python
class EcsRunTaskOperator_withRetry(EcsRunTaskOperator):
    _KNOWN_ERRORS = ["Capacity is unavailable at this time."]
    _MAX_NUM_ATTEMPTS = 2
    _RETRY_DELAY_SECS = 15

    def _start_task(self) -> None:
        # RunTask; retry known recoverable launch failures.
        ...

    def _should_retry_error(exception: Exception) -> bool:
        if isinstance(exception, EcsTaskFailToStart):
            return any(msg in exception.message for msg in [
                "network interface provisioning",
                "ResourceInitializationError",
                "Timeout waiting for EphemeralStorage provisioning to complete",
            ])
        return False

    @AwsBaseHook.retry(_should_retry_error)
    def _check_success_task(self) -> None:
        super()._check_success_task()
```

Listing 3. Sanitized bounded reliability semantics for ECS/Fargate launch and post-launch provisioning failures.

For capacity errors, the explicit retry budget adds at most 15 seconds before the second launch attempt. If p is the probability of a transient launch failure, one retry improves success probability from $1 - p$ to $1 - p^2$. This is not magic reliability. It is a practical ceiling on a boring class of flakes.

5.5. **Workload families.** Four workload families use the pattern.

- **Near-source collectors** run close to exchange endpoints. Freshness is the binding constraint.
- **Transformation and archival jobs** run where the relevant storage is local and cheap to access.
- **Local QA and cross-region checks** are modeled explicitly. Local checks stay in region; cross-region consistency checks pay their cost in a visible workflow.
- **Scheduled observers** run in the region of the systems they observe.

The common rule is simple: placement is part of the workflow's identity, not a hidden runtime preference.

6. Measuring freshness

Network RTT is easy to measure and too easy to overrate. What the platform delivers is usable knowledge. We therefore measure *knowledge latency* with timestamps carried through the data path.

6.1. **Instrumentation.** The ingestion path records the timestamp when a data point reaches collector memory, for example an `end_download_timestamp`. The storage path records the timestamp when the record becomes query-visible, for example a `knowledge_timestamp`. The difference is the metric:

$$K = t_{\text{knowledge}} - t_{\text{end\_download}}.$$

This can be computed with ordinary database queries. It does not require a distributed tracing system everywhere, although tracing can make root-cause analysis easier.

6.2. **Exchange API timing.** Before measuring full pipeline freshness, we measured the placement tax directly against a public exchange endpoint. A representative `curl -w` breakdown for the Crypto.com Exchange `public/get-book` endpoint is shown in Listing 4. The measurement follows the standard curl timing method [19, 12].

```
time_namelookup:    0.003162s
time_connect:       0.004825s
time_appconnect:    0.049827s
time_pretransfer:   0.049995s
time_starttransfer: 0.224340s
----------
time_total:         0.224417s
```

Listing 4. Representative curl timing breakdown against a Crypto.com order-book endpoint.

Across 10 attempts from each caller region, Tokyo was faster than Singapore by about 12.7 ms for this endpoint in this measurement window. Table 2 reports the averages.

Caller region	Average total time	Attempts
Tokyo	0.1880250 s	10
Singapore	0.2007697 s	10

Table 2. Crypto.com API response time by caller region. The 12.7 ms delta is endpoint- and time-window-specific, but it shows that placement is measurable even before pipeline overhead is added.

The difference is smaller than raw inter-region RTT because API timing includes TLS, server-side work, and provider behavior. That is fine. The goal is not to isolate physics in a lab. The goal is to measure the latency the pipeline actually pays.

6.3. **Network floor.** We also measured the network floor between a Tokyo caller and a Singapore-hosted endpoint. The ICMP RTT summary and representative TCP throughput are shown in Table 3.

Metric	Min	Mean	Max / observed
Ping RTT, Tokyo to Singapore	68.7 ms	68.76 ms	69.4 ms
TCP throughput, Tokyo–Singapore	–	–	190 Mbit/s over 10.1 s

Table 3. Representative network baseline across regions. These numbers are not universal; they are a sanity check that region choice has a visible floor.

7. Evaluation

We evaluate the pattern along three questions: does geography show up, does the scheduler survive the volume, and are known launch failures contained.

7.1. **E1. Geography shows up.** The API timing in Table 2 shows a 12.7 ms average delta between two caller regions for one exchange endpoint. The RTT baseline in Table 3 shows about 68.8 ms across the Tokyo–Singapore path. The specific values will change with provider, endpoint, time, routing, and cloud region. The durable point is that geography is visible before the application does any work. A pipeline that ingests, validates, transforms, and commits data can only add to that baseline.

7.2. **E2. A single scheduler can carry regional volume.** We computed DAG inventory from the repository filenames in `datapull/airflow/dags/datapull`. Including pre-production and test DAG files, the repository contains 69 region-tagged DAGs: 32 tagged for Tokyo and 37 tagged for Europe. Counting periodic pre-production schedules and excluding backfills and ad-hoc workflows, the implied scheduled volume is 29,401 triggers per day. Table 4 breaks down the inventory.

Region	Region-tagged DAGs	1-min	10-min	Triggers/day
Tokyo	32	13	10	20,164
Europe	37	6	4	9,237
Total	69	19	14	29,401

TABLE 4. Region-tagged DAG inventory and implied scheduled trigger volume. The trigger count uses periodic pre-production schedules; backfills, ad-hoc jobs, and tests are excluded from the daily trigger total.

This is the operational win. Regional execution does not require regional schedulers. The scheduler stays centralized while execution moves to the region where work belongs.

7.3. **E3. Launch failures are bounded.** The retry wrapper contains two common classes of cloud flakiness. Temporary capacity failures at `RunTask` receive a bounded retry with a 15-second delay. Known post-launch provisioning failures are retried through the Airflow AWS hook retry mechanism. Unknown failures remain failures.

This matters more than it sounds. Without this layer, cloud substrate noise leaks into workflow health. With it, a familiar failure class becomes a bounded launch delay. The policy is conservative: retry the failures we have seen and understand, fail cleanly on the rest.

7.4. **What this does not prove.** The evaluation is a production case study. It does not prove that this pattern dominates all alternatives. It does not claim exchange-grade co-location. It does not provide controlled before/after pipeline latency across every workload family. What it does show is that the architecture names the right seams: placement before scheduling, regional execution at launch, bounded retry below the scheduler, and freshness as the metric that matters.

8. GENERALIZATIONS

The pattern does not depend on Airflow or ECS/Fargate.

Argo Workflows. A single Argo controller can own workflow state. A required region label on the Workflow custom resource can implement the placement contract, validated by an admission webhook. Per-region node pools can scale from zero. Retry behavior can be expressed through Argo retry strategies and executor-level classification of pod failure reasons [17, 15, 16, 14].

Temporal. Temporal can keep a central workflow history while routing activities to regional task queues. The placement contract becomes the mapping from workflow identity to task queue. Bounded launch retries move into activity retry policies and worker-level error classification.

Airflow with KubernetesPodOperator. The same approach works with a single Airflow deployment and regional Kubernetes clusters. The placement contract selects the target cluster context, namespace, node pool, storage class, and retry policy. The handoff moves from ECS RunTask to Kubernetes pod creation.

The shared idea is the same in each case. Keep control state centralized. Make placement explicit. Let regional execution scale elastically. Retry known substrate failures below the scheduler.

9. Related work

Workflow orchestration and container execution. Airflow provides mature scheduling, dependency management, and operational visibility [4]. The Amazon provider exposes `EcsRunTaskOperator` for launching ECS tasks from Airflow [5, 6]. ECS RunTask and Fargate supply the elastic execution substrate [1, 2]. Our contribution is the placement and reliability discipline around that integration, not a new scheduler.

Geo-distributed streaming and replication. Kafka and MirrorMaker address stream transport and cross-cluster replication [9, 8, 7]. Geo-distributed stream-processing work such as SWAN and TTL-based aggregation studies placement and coordination across regions [22, 18]. Our work sits one level up: batch, streaming, QA, archival, and observer workflows share a scheduler while launching regionally placed containers.

Edge and near-source compute. Edge computing argues for moving computation closer to data sources and users [20]. Near-exchange compute is a specific, finance-shaped instance of that idea. We do not claim hard real-time guarantees or physical exchange co-location. We show how cloud workflow infrastructure can preserve the placement intent without multiplying schedulers.

Hardware-assisted ingestion. FPGA and NIC-based systems, including Corundum and data-center accelerator cards, push the near-source idea closer to the wire [13, 11, 3]. Those systems are complementary. A centralized scheduler can still launch and supervise hardware-adjacent pipelines, provided the placement contract includes the relevant device and region constraints.

10. Limitations and future work

10.1. **Limitations.**

Cloud regions are not exchange co-location. Running in Tokyo is not the same as sitting inside an exchange facility. The pattern reduces avoidable cloud-region distance. It does not defeat physics or replace specialized market-access infrastructure.

Filename contracts are useful, but not beautiful. A filename-based placement contract is easy to review and hard to hide. It is also brittle once the number of regions, environments, and workflow families grows. A typed manifest or validated workflow resource would give better schema evolution, richer validation, and clearer tooling.

The retry list is empirical. Bounded retries depend on known error patterns. New cloud failure modes must be classified and added deliberately. Retrying everything would be dishonest. Retrying nothing leaves avoidable failure on the table.

Full pipeline latency still needs broader reporting. The system carries the timestamps needed for *knowledge latency*, and the paper reports representative network and inventory measurements. A stronger future evaluation would report P50/P95/P99 knowledge latency by workflow family, region, and exchange endpoint over a longer window.

10.2. **Future work.**

Typed placement manifests. Move from filename parsing to a typed placement manifest with explicit region, storage, network, device, and retry fields. The manifest can be validated in CI and rendered into both Airflow and non-Airflow substrates.

Autoscaling both planes. The control plane already benefits from Kubernetes scaling primitives. The execution plane can go further with event-driven scaling, regional capacity-aware placement, and explicit degradation policies when a region has no available Fargate capacity.
Failover placement. Some workflows may tolerate failover to a secondary region. That should be explicit in the placement contract, with freshness penalties visible in the workflow definition rather than hidden in runtime code.
Hardware-adjacent pipelines. The same control-plane pattern can launch collectors that run near FPGA/NIC-accelerated ingest paths. The scheduler still does not need to know packet mechanics. It only needs a placement contract rich enough to describe where the work belongs.

Key takeaways

- Do not multiply schedulers just to move work closer to data. Keep workflow state centralized unless regional scheduler autonomy is truly needed.
- Make placement reviewable. A region hidden in an operator argument will drift. A placement contract forces the decision into code review.
- Retry cloud launch flakes below the scheduler, with budgets. The scheduler should not be the first place that learns an ENI took too long to appear.
- Measure freshness, not only network RTT. Users care when the system knows, not when a packet arrived halfway through the pipeline.

11. Conclusion

Centralized orchestration and regional execution separate two concerns that are too often fused. The scheduler should own workflow state, dependency ordering, and operational visibility. The execution plane should run work in the region where the work belongs. A deterministic placement contract makes that decision explicit. Bounded reliability semantics keeps familiar cloud-launch failures from becoming workflow drama.

For the Causify Trading Platform, the pattern gave a practical middle path. We kept one Airflow scheduler, launched containers across regions through ECS/Fargate, encoded placement in workflow identity, and measured the geography tax directly. The result is not a universal recipe. It is a useful discipline: keep control simple, move execution close, and measure freshness where it becomes real.

References

[1] Amazon Web Services (ECS). Runtask, amazon elastic container service api reference. Documentation. Accessed 2026-03-23. URL: https://docs.aws.amazon.com/AmazonECS/latest/APIReference/API_RunTask.html.
[2] Amazon Web Services (Fargate). Amazon ecs task networking options for fargate. Documentation. Accessed 2026-03-23. URL: https://docs.aws.amazon.com/AmazonECS/latest/developerguide/fargate-task-networking.html.
[3] AMD. Amd alveo u250 data center accelerator card. Website. Accessed 2026-03-23. URL: https://www.amd.com/en/products/accelerators/alveo/u250/a-u250-a64g-pq-g.html.
[4] Apache Airflow. Apache airflow documentation. Website. Accessed 2026-03-23. URL: https://airflow.apache.org/docs/.
[5] Apache Airflow Amazon Provider. Amazon elastic container service (ecs) operator (ecsruntaskoperator). Documentation. Accessed 2026-03-23. URL: https://airflow.apache.org/docs/apache-airflow-providers-amazon/stable/operators/ecs.html.
[6] Apache Airflow Providers. airflow.providers.amazon.aws.exceptions (ecstaskfailtostart, ecsoperatorerror). Documentation. Accessed 2026-03-23. URL: https://airflow.apache.org/docs/apache-airflow-providers-amazon/stable/_api/airflow/providers/amazon/aws/exceptions/index.html.
[7] Apache Kafka MirrorMaker. Mirrormaker 2 configuration (mirrormaker configs). Documentation. Accessed 2026-03-23. URL: https://kafka.apache.org/40/configuration/mirrormaker-configs/.
[8] Apache Kafka Operations. Geo-replication (cross-cluster data mirroring). Documentation. Accessed 2026-03-23. URL: https://kafka.apache.org/40/operations/geo-replication-cross-cluster-data-mirroring/.

[9] Apache Kafka Project. Apache kafka documentation. Website. Accessed 2026-03-23. URL: https://kafka.apache.org/documentation/.

[10] Debopam Bhattacherjee et al. A bird's eye view of the world's fastest networks. In *Proceedings of the ACM Internet Measurement Conference (IMC)*, 2020. Accessed 2026-03-23. URL: https://bdebopam.github.io/papers/imc2020-hft.pdf.

[11] Corundum Contributors. Corundum: Open source fpga-based nic and platform for in-network compute. GitHub. Accessed 2026-03-23. URL: https://github.com/corundum/corundum.

[12] Crypto.com. Crypto.com exchange api v1 (rest & websocket) documentation. Documentation. Accessed 2026-03-23. URL: https://exchange-docs.crypto.com/exchange/v1/rest-ws/index.html.

[13] Alex Forencich, Alex C. Snoeren, George Porter, and George Papen. Corundum: An open-source 100-gbps nic. In *Proceedings of the IEEE International Symposium on Field-Programmable Custom Computing Machines (FCCM)*, 2020. Accessed 2026-03-23. URL: https://cseweb.ucsd.edu/~snoeren/papers/corundum-fccm20.pdf.

[14] KEDA Project. Keda documentation: Scaling deployments and statefulsets. Documentation. Accessed 2026-03-23. URL: https://keda.sh/docs/2.19/concepts/scaling-deployments/.

[15] Kubernetes Autoscaling. Horizontal pod autoscaling. Website. Accessed 2026-03-23. URL: https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/.

[16] Kubernetes Cluster Administration. Node autoscaling (cluster autoscaler). Website. Accessed 2026-03-23. URL: https://kubernetes.io/docs/concepts/cluster-administration/node-autoscaling/.

[17] Kubernetes Documentation. Kubernetes components. Website. Accessed 2026-03-23. URL: https://kubernetes.io/docs/concepts/overview/components/.

[18] D. Kumar et al. A ttl-based approach for data aggregation in geo-distributed streaming analytics. In *Proceedings (preprint/pdf)*, 2019. Accessed 2026-03-23. URL: https://groups.cs.umass.edu/wp-content/uploads/sites/3/2019/12/A-TTL-based-Approach-for-Data-Aggregation-in-Geo-distributed-Streaming-Analytics.pdf.

[19] Joseph Scott. Timing details with curl. Blog post. Accessed 2026-03-23. URL: https://blog.josephscott.org/2011/10/14/timing-details-with-curl/.

[20] Weisong Shi and Schahram Dustdar. The promise of edge computing. *Computer*, 2016. Accessed 2026-03-23. URL: https://www.infosys.tuwien.ac.at/Staff/sd/papers/Zeitschriftenartikel_2016_SD_The_Promise.pdf.

[21] Ankit Singla et al. The internet at the speed of light. In *HotNets XIII*, 2014. Accessed 2026-03-23. URL: https://conferences.sigcomm.org/hotnets/2014/papers/hotnets-XIII-final111.pdf.

[22] Won Wook Song et al. Swan: Wan-aware stream processing on geographically-distributed clusters. In *Proceedings of APSys*, 2022. Accessed 2026-03-23. URL: https://wonook.github.io/papers/apsys22-song-swan.pdf.

About the Authors

Shayan Ghasemnezhad is Infrastructure and DevOps Lead at Causify AI, Turin, Italy. His research interests include platform engineering and cloud solutions architecture, production AI/ML systems, and secure, reliable infrastructure operations. Ghasemnezhad received his Bachelor's degree in Digital Management from Ca' Foscari University of Venice. Contact him at shayan@causify.ai.

Giacinto Paolo (GP) Saggese is Co-founder and CTO at Causify AI, Potomac, Maryland, USA. His research interests include machine learning, fintech, and the commercialization of research. Saggese received his Ph.D. degree in Electrical and Computer Engineering from the University of Illinois Urbana-Champaign. He is an Adjunct Professor at the University of Maryland, College Park. Contact him at gp@causify.ai.

Paul Smith is Co-founder and Chief Scientist at Causify AI, Cary, North Carolina, USA. His research interests include causal inference, Bayesian modeling, and time-series prediction. Smith received his Ph.D. degree in Mathematics from UCLA. Contact him at paul@causify.ai.

Heanh Sok is Software Engineer at Causify AI, College Park, Maryland, USA. His research interests include big data systems, distributed systems, and AI/ML engineering and operations. Sok received his Master's degree in Data Science from the University of Maryland, College Park. Contact him at h.sok@causify.ai.